

ClashGPT: An Autonomous Clash Royale Agent

Using Computer Vision and Proximal Policy Optimization

Ezra Gilchrest

Computer Systems Lab, Thomas Jefferson High School for Science and Technology

Instructor: Dr. Selma Yilmaz | Period 7

May 2026

GitHub Repository: <https://github.com/EzraGil08/ClashGPT-Autonomous-Clash-Royale-Artificial-Intelligence>

Demo Video Playlist: <https://www.youtube.com/playlist?list=PL32XXBfd5bVLy0m9os1xYXtizypVCz>

Abstract

Clash Royale is a competitive real-time mobile strategy game in which two players deploy troops and spells from a hand of eight cards to destroy each other's towers within three minutes. The game presents significant challenges for autonomous agents: there is no official API, actions must be taken in real time under elixir constraints, and the decision space spans tens of thousands of possible card-placement combinations per game. No prior open-source Clash Royale bot combines a full computer vision pipeline with a reinforcement learning agent trained on live gameplay.

This paper describes ClashGPT, an autonomous Clash Royale agent built on a pipeline of screen capture, dual YOLOv11s object detection, SORT-based multi-object tracking, pixel-level elixir and tower health detection, template-matching hand recognition, and a MaskablePPO reinforcement learning policy. The agent observes a 202-dimensional state vector encoding troop positions and types, the player hand, elixir, and tower health percentages for all six towers. It selects from 49 discrete actions corresponding to every legal card placement plus a wait action, with invalid actions masked at every step. The reward function combines dense per-frame tower health signals with sparse event bonuses for tower destruction, wins, and contextual plays such as deploying cards behind a Giant. After approximately 100 training games against a fixed opponent deck, the agent achieved a 10% win rate against human opponents ranging from 4,000 to 13,500 trophies, winning one game 2-1 after losing a tower. This result surpasses the 0% win rate of a deterministic scripted baseline against the same opponents. The project is fully open-source and reproducible from the provided GitHub repository and README.

I. Introduction

Clash Royale, developed by Supercell, is a competitive real-time strategy game that has accumulated over 100 million downloads worldwide. Players choose eight cards from a pool of over 121 unique options, forming a deck with an average elixir cost that determines how quickly cards can be cycled. During a three-minute match, both players deploy units and spells onto a shared arena, attempting to destroy the opponent's princess towers and, ultimately, the king tower. The player who destroys the most towers wins; if neither player reaches the king tower, the player with more surviving towers wins on points.

The game presents a rich challenge for artificial intelligence research. Unlike turn-based games such as chess or Go, Clash Royale is played in continuous real time, demanding decisions on the order of seconds. There is no official API or simulator: all game state must be inferred from the raw pixel output of an Android emulator. The action space is large, pairing each of the eight deck cards with dozens of possible arena tile placements. Card interactions are highly strategic: a Giant absorbs tower fire while a Witch summons skeletons to deal damage behind it, and a Graveyard spell spawning skeletons directly on a tower is devastating only when deployed during an active push.

The goal of this project is to build the first fully open-source Clash Royale bot that combines a complete computer vision pipeline with a trained reinforcement learning agent. The hypothesis is that a MaskablePPO agent receiving dense, informative rewards derived from per-frame tower health changes will learn meaningful game strategies within a tractable number of real-time training games. The primary success criterion is winning at least one game in ten against a human opponent.

The remainder of this paper is organized as follows. Section II reviews related work and the gap in prior solutions. Section III discusses the broader applications of this research. Section IV describes the full system architecture, all algorithms, and implementation details. Section V presents quantitative and qualitative results. Section VI discusses limitations. Section VII concludes, and Section VIII outlines future work.

II. Background and Related Work

A. AI in Competitive Games

The challenge of building AI agents for competitive real-time games has attracted significant research effort. DeepMind's AlphaStar defeated professional StarCraft II players in 2019 using a combination of reinforcement learning, imitation learning from human replays, and a multi-agent training curriculum [1]. OpenAI Five defeated world champion Dota 2 teams in 2018 using Proximal Policy Optimization (PPO) with a large distributed training infrastructure and a shaped reward signal derived from game-internal state [2]. Both systems had access to structured game state through official APIs, allowing the policy to receive clean observations without a computer vision layer.

Clash Royale shares core challenges with these games: partial observability (the opponent's hand is hidden), time pressure, and complex emergent interactions between units. However, it differs in two important respects. First, there is no API: every piece of state must be recovered from pixels. Second, the training environment cannot be accelerated beyond real-time gameplay speed, since the agent must interact with a live Android emulator running on a PC.

B. Prior Clash Royale Bots

Three prior approaches to Clash Royale automation were identified and evaluated.

The first is a basic scripted bot (commercially available as 'ClashGPT' prior to this project's existence) that automates card plays using fixed timing and hardcoded placement coordinates. It has no model of game state, cannot adapt to opponent cards, and is trivially exploitable. Its win rate against skilled human opponents is 0%.

The second is Ombarus [3], a YouTube project that uses computer vision to detect troop positions and elixir, then plays cards using hardcoded decision logic. While it demonstrates that vision-based state reading is feasible, it cannot distinguish ally from enemy troops, has no threat detection, and is not publicly reproducible.

The third is a student project by Junior Builds [4] that applies Q-learning to a state vector derived from a third-party Roboflow detection model. It lacks ally/enemy classification, has no tower health detection (making the reward signal sparse to the point of being only win/loss), and uses epsilon-greedy Q-learning, which does not generalize well to the large action space of Clash Royale.

C. Gap in Existing Work

No prior open-source Clash Royale agent combines: (1) custom-trained object detection models for both troops and spells, (2) SORT-based tracking with ally/enemy team separation, (3) pixel-level tower health detection enabling dense per-frame rewards, (4) a full structured state vector encoding troops, hand, elixir, and tower health, and (5) a trained reinforcement learning policy using PPO. Table 1 summarizes the comparison.

Feature	Basic Bot	Ombarus	Junior Builds	ClashGPT (This Work)
Elixir tracking	No	Yes	Yes	Yes
Ally/enemy classification	No	No	No	Yes
Tower health detection	No	No	No	Yes
Structured game state	No	Partial	Partial	Yes
Reinforcement learning	No	No	Yes	Yes
Custom YOLO models	No	No	No	Yes
Publicly reproducible	No	No	No	Yes

Table 1. Feature comparison of Clash Royale automation approaches.

III. Applications

The techniques developed in this project have applications beyond Clash Royale.

- Real-time game AI: The full pipeline of screen capture, object detection, tracking, structured observation, and masked policy optimization is directly transferable to any game that lacks an API, including mobile games, older titles without modding support, and commercial games where API access is restricted.
- Pixel-based reinforcement learning: The approach of reading all game state from pixels rather than from an API is relevant to any domain where a structured interface is unavailable, including legacy industrial systems, accessibility tools for navigating graphical interfaces, and automated software testing.
- Strategic AI research: Clash Royale's combination of resource management (elixir), combinatorial strategy (deck building), and real-time execution makes it a useful benchmark for studying how RL agents learn to balance long-horizon planning with reactive decision-making.
- Esports analysis: A trained agent that understands troop positioning, card synergies, and elixir efficiency could serve as a coaching tool, analyzing replays to identify suboptimal plays.

IV. Methods

A. System Architecture

The full data flow is as follows. At each timestep the system captures a screenshot of the BlueStacks Android emulator window using the mss library [5]. This frame is passed simultaneously to five subsystems: (1) dual YOLO inference for troop and spell detection, (2) SORT-based multi-object tracking with team classification, (3) pixel-level elixir detection, (4) template-matching hand detection, and (5) pixel-scanning tower health detection. The outputs of all five subsystems are combined into a 202-dimensional observation vector. This vector is passed to the MaskablePPO policy, which selects one of 49 discrete actions. The chosen action is executed via PyAutoGUI mouse clicks on the emulator window, and the cycle repeats, as illustrated in Figure 4.

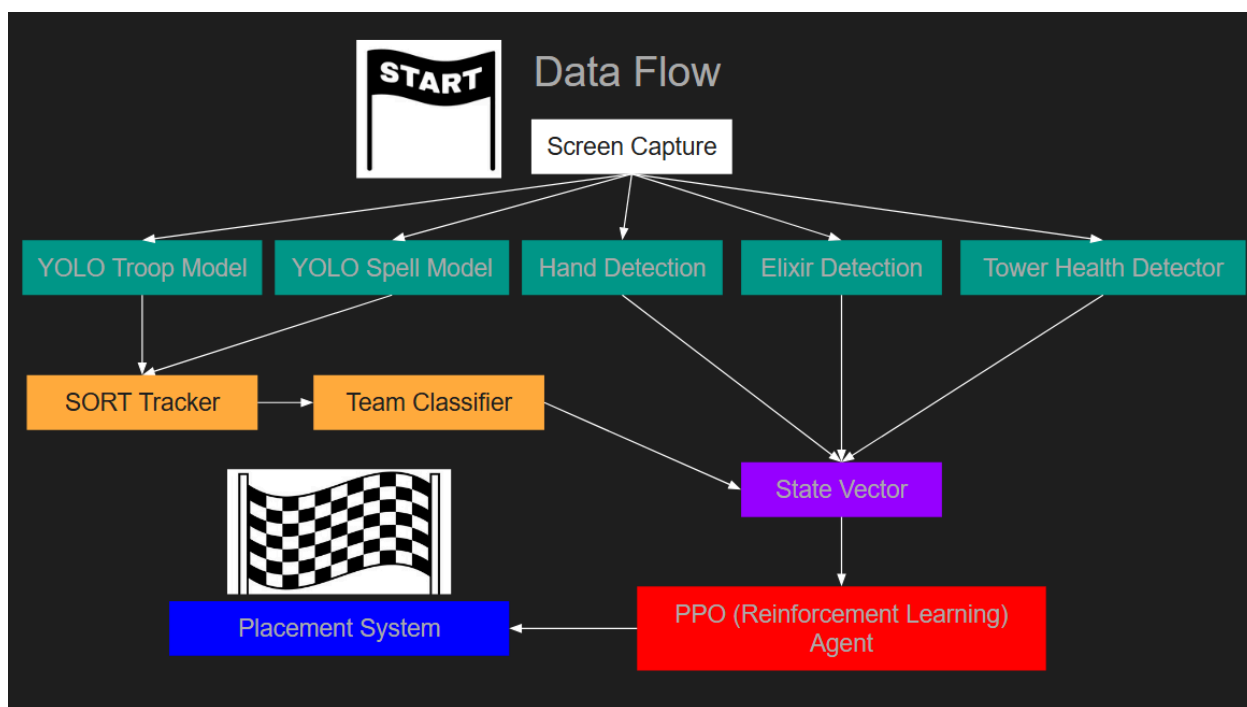


Figure 4. System architecture and data flow. At each timestep, a screen capture is passed to five parallel subsystems whose outputs are combined into a state vector, which the MaskablePPO agent uses to select a card placement action.

B. Screen Capture and Calibration

Screen capture is performed using mss, which achieves approximately 40-50 frames per second on the test hardware (Intel Core i7, 15.6 GB RAM). Calibration is performed once per session: the user moves the mouse to the top-left corner of the BlueStacks window for three seconds, then to the bottom-right corner for three seconds. The capture region is recorded as (left, top, width, height) in screen coordinates.

All subsequent pixel coordinate lookups are performed relative to an anchor point returned by `get_top_corner()`, which scans the first captured frame for the pixel color BGR(72, 30, 24) with a

tolerance of 5, corresponding to a unique dark corner pixel of the game UI. This anchor is computed once at the start of each episode and stored as a (cx, cy) offset applied to all fixed pixel coordinates. This approach handles any window position or resolution without recalibration.

C. YOLO Object Detection

Two separate YOLOv11s models are trained and run in parallel on every frame (with a configurable skip rate of every 3 frames to reduce GPU load during training).

LBGG (Large Bodies, Ground, Giants) is trained to detect the eight player deck units (Giant, Bowler, Witch, Guards, Minions), spawned units (Skeleton, Goblin, Spear Goblin), and opponent deck units (Princess, Ice Spirit, Rocket, Goblin Barrel, Knight, Log, Cannon). SARG (Spells, Arrows, Rockets, Graveyard) is trained to detect area-effect spells and projectiles including Arrows, Snowball, and Graveyard.

Data Collection

Gameplay was recorded on BlueStacks at native resolution. Video was manually clipped to remove sections without active unit motion, then parsed into frames at 1 frame per second for troop footage (yielding 292 frames) and 5 frames per second for spell footage (yielding 576 frames). An additional two videos were recorded after initial training to address low-accuracy classes. All frames were labeled manually using Label Studio [6] with tight bounding boxes. The dataset was split 80% training, 15% validation, and 5% test.

Training Configuration

Both models were fine-tuned from pretrained YOLOv11s weights using the Ultralytics library (version 8.3.202) [7]. Training hyperparameters: 75 epochs, batch size 8, learning rate 0.001, image size 640. Training was performed on the Intel Core i7 CPU. Both models use a confidence threshold of 0.35 and an IoU NMS threshold of 0.45 at inference time.

Detection Results

On a held-out test set of 10 game screenshots, the LBGG troop model achieved 85% mean average precision (mAP50) and the SARG spell model achieved 92% mAP50. The higher spell accuracy reflects the visually distinctive appearance of spell effects compared to the greater visual similarity between ground unit types at small screen sizes.

D. SORT Tracking and Team Classification

Detections from both YOLO models are merged into a single list and passed to a custom SORT (Simple Online and Realtime Tracking) tracker implemented in tracker.py. SORT combines a 7-state Kalman filter (tracking center x, center y, scale, aspect ratio, and their velocities) with Hungarian matching using IoU as the cost metric [8].

A key novelty of this implementation is the use of two independent track pools: one for ally units and one for enemy units. This prevents a common failure mode in single-pool trackers where a friendly troop crossing near an enemy troop can steal the enemy's track ID, corrupting the team label. Once a track is assigned to a team pool, it never crosses to the other pool.

Team assignment uses the following logic. For units that are definitively ally-only (Giant, Bowler, Witch, Guards, Minions) or enemy-only (units prefixed with e_ in the YOLO class names), the team is assigned directly from the class name. For ambiguous spawned units (Skeleton, Goblin, Spear Goblin) that can appear on either side, team is determined by arena position: normalized $y > 0.5$ (lower half of screen, closer to the ally side) is classified as ally, and normalized $y \leq 0.5$ is classified as enemy.

Each track also implements a class consistency vote: the majority class label across all matched detections is used as the track's class, preventing a brief misclassification from permanently changing a track's identity. Tracks must accumulate $\text{min_hits} = 2$ matched detections before being returned to the observation builder, suppressing spurious single-frame detections. Tracks that go unmatched for more than $\text{max_age} = 8$ frames are pruned. Figure 1 shows the full CV pipeline operating during two evaluation games.

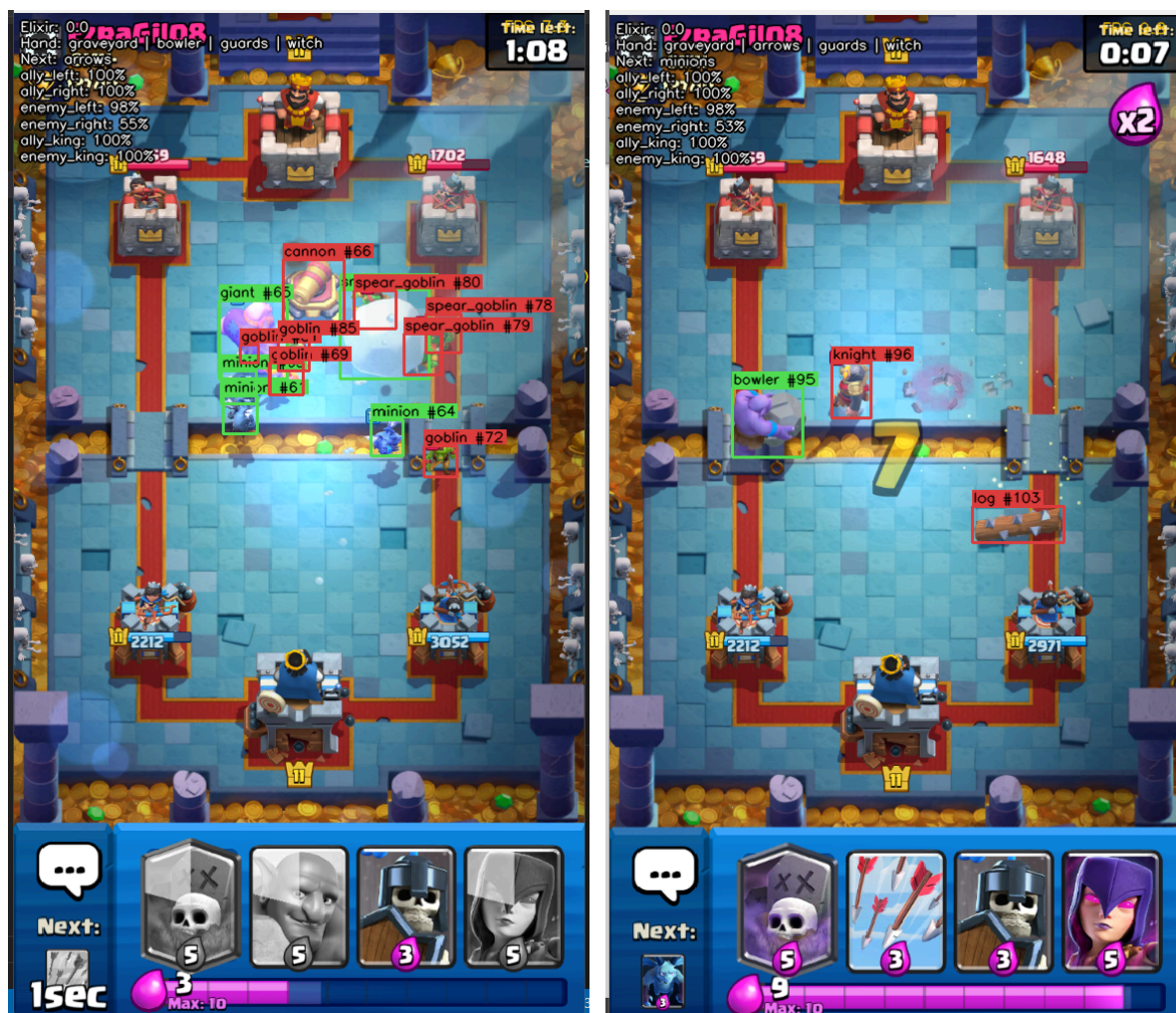


Figure 1. Live computer vision overlay during two evaluation games. Bounding boxes show SORT-tracked troop and spell detections with ally/enemy classification, alongside real-time elixir, hand, and tower health readings.

E. Elixir Detection

Elixir is read from the game's elixir bar using HSV color matching on individual pixels. The algorithm first checks a single known pixel at position $(538 + cx, 978 + cy)$ for the full-bar color (HSV range $[143,190,150]$ to $[153,240,230]$) as an early-exit optimization. If the bar is not full, the algorithm scans ten sampled x-positions across the bar, counting full-bar pixels (each worth 1.0 elixir) and partial-bar pixels matching a lighter blue color (each worth a fractional amount based on their position within the bar segment). The result is a float in $[0.0, 10.0]$ rounded to two decimal places.

A spike suppression filter is applied: because elixir can only regenerate at approximately 0.3 per second in-game, any upward jump of more than 0.15 above the previous reading is clamped to $\text{previous} + 0.15$. This eliminates color artifacts from animations that would otherwise cause the elixir to appear to jump by several points in a single frame.

F. Hand Detection

The four cards currently in the player's hand are identified using normalized cross-correlation template matching (OpenCV `TM_CCOEFF_NORMED`) [9]. A cropped region spanning the hand area (y: 820 to 951, x: 124 to 541, relative to the anchor corner) is compared against stored grayscale template images of all eight deck cards plus an 'empty' placeholder. The top four matches are selected subject to a minimum horizontal separation of 50 pixels, preventing the same card image from matching multiple slots. The matches are sorted by x-coordinate to determine left-to-right slot order.

The next card (shown as a smaller preview to the left of the hand) is detected using a separate set of mini card templates matched against a smaller region (y: 930 to 986, x: 29 to 75).

G. Tower Health Detection

Tower health percentages are read by scanning horizontal strips of pixels across the health bar regions for each of the six towers (left and right princess towers for each player, plus both king towers). The pixel positions of each bar are hard-coded relative to the anchor corner, with reference coordinates measured from a calibrated game frame using [pixspy.com](https://pyspy.com).

Ally tower health bars use a gold/orange color palette (BGR colors approximately $[205,164,97]$, $[255,210,114]$, and $[250,208,114]$). Enemy tower health bars use a purple/blue palette (BGR approximately $[80,38,204]$, $[94,38,226]$). King tower bars become active only after one of that side's princess towers is destroyed.

The scanner walks each bar pixel by pixel from left to right, counting health-colored pixels until it encounters a terminate color (the color of the bar background when the tower has taken damage) or an unexpected color. If an unexpected color is encountered on a princess bar, the algorithm polls the corresponding king bar as confirmation: if the king bar shows health, the princess tower is likely dead, and this must be confirmed over `KING_CONFIRM_FRAMES = 3` consecutive frames before the tower is permanently locked to 0% health. This multi-frame confirmation prevents single noisy frames from prematurely declaring a tower dead at episode start.

A separate HP noise filter in `env.py` suppresses tower health deltas smaller than 0.5 percentage points, treating them as color detection jitter rather than real damage events.

H. Observation Vector

The observation vector is a flat array of 202 float32 values in $[0, 1]$, constructed by `observation.py`. The layout is as follows.

Indices	Description	Size
0 to 149	Up to 30 troops, 5 features each: normalized x center, normalized y center, is_ally flag, normalized class index, normalized track age	150
150 to 185	4 hand slots, each one-hot encoded over 9 card classes (8 deck cards + empty)	36
186 to 194	Next card one-hot over 9 card classes	9
195	Current elixir divided by 10	1
196 to 201	Six tower health percentages divided by 100 (ally left, ally right, enemy left, enemy right, ally king, enemy king)	6

Table 2. Observation vector layout (202 floats total).

If more than 30 troops are tracked simultaneously, non-skeleton troops are prioritized and older tracks are dropped first. This is because skeletons (spawned by the Witch and Graveyard) can briefly fill the board during heavy pushes, and their individual positions are less strategically significant than those of high-value units.

I. Action Space and Masking

The action space consists of 49 discrete actions defined in `constants.py`. Actions 0 through 47 correspond to specific (card, placement coordinate) pairs, grouped in contiguous blocks per card. Action 48 is the WAIT action. The number of placement coordinates per card ranges from 2 (Graveyard, which is only useful in a small number of locations) to 8 (Arrows and Snowball, which can be placed anywhere on the arena).

At each timestep, an action mask is computed by `get_action_mask()` in `actions.py`. An action is legal if and only if its card is present in the current hand AND the player's elixir is greater than or equal to that card's cost. The WAIT action is always legal. MaskablePPO from the `sb3-contrib` library [10] uses this mask to zero out the logits of illegal actions before sampling, ensuring the agent never wastes a step attempting to play a card it cannot afford.

J. Reward Function

The reward function in `reward.py` combines dense per-frame signals with sparse event-triggered bonuses. The design goal is to give the agent meaningful credit for damage dealt to enemy towers on the same step that the damage is read, rather than waiting for a terminal win/loss signal that would arrive hundreds of steps later.

Component	Value	Trigger
Enemy tower HP damage	+0.01 per 1%	Each frame an enemy princess tower loses health
Ally tower HP damage	-0.02 per 1%	Each frame an ally princess tower loses health
Positional pressure (ally)	+0.003 per troop	Each ally troop with normalized $y < 0.5$ (enemy half)
Positional pressure (enemy)	-0.003 per troop	Each enemy troop with normalized $y > 0.5$ (ally half)
Enemy tower destroyed	+2.0	Princess tower HP transitions from > 0 to 0
Ally tower destroyed	-2.0	Ally princess tower HP transitions from > 0 to 0
Win	+5.0	Episode terminates with all enemy towers at 0
Loss	-5.0	Episode terminates with all ally towers at 0
Spell missed	-0.3	Spell played with no enemy HP drop or troop kill
Play at low elixir (< 3)	-0.15	Any card played when elixir < 3
Play at high elixir (≥ 7)	+0.05	Any card played when elixir ≥ 7
Wait with cards available, elixir ≥ 8	-0.05	Agent waits despite having playable cards and high elixir
Forced wait, elixir < 4	+0.02	Agent cannot afford any card and elixir is below 4 (building toward a play)
Card played behind Giant	+0.4 + 0.1 * supporting allies	Any non-Giant card played into a lane with a friendly Giant
Graveyard into active push	+1.5	Graveyard played while ally troops are in the enemy half of that lane

Table 3. Reward function components.

The asymmetric weighting of ally tower damage (-0.02) versus enemy tower damage (+0.01) reflects the reality that allowing a tower to fall is harder to recover from strategically than the elixir cost of a single card.

The contextual bonuses (behind Giant, Graveyard push) encode two key heuristics of the Giant-Graveyard deck: placing support units behind a tanking Giant maximizes their damage, and deploying Graveyard skeletons directly onto a tower while the opponent is distracted by a push is the primary win condition of this archetype.

K. Gymnasium Environment

The full environment is implemented as a Gymnasium [11] `env.py` class `CREnv`. The `observation_space` is a `Box` of shape (202,) with all values in $[0, 1]$. The `action_space` is `Discrete(49)`. The environment wraps the entire detection and reward pipeline; the caller only needs to call `reset()` at the start of a game and `step(action)` at each decision point.

YOLO inference is skipped every `YOLO_SKIP_FRAMES = 3` frames (i.e., YOLO runs on frame 0, 3, 6, ...) to reduce GPU load. On skipped frames, the tracker is updated with an empty detection list, which allows existing tracks to coast on their Kalman-predicted positions. Elixir and tower health are read every frame since they use CPU-only pixel scanning.

Episodes terminate automatically when all towers on either side reach 0% health. The caller can also invoke `force_done(won=True/False)` to signal a game end that was detected externally (for example, by the `BannerDetector` in `train.py`, which uses template matching to detect the end-of-game crown banner). Figure 2 and Figure 3 illustrate two of the contextual reward components in practice.

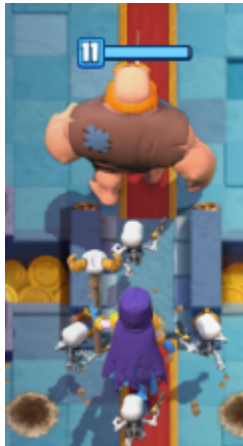


Figure 2. The agent deploying a Witch behind a Giant during an evaluation game, triggering the +0.4 contextual reward for supporting a tank push.

```
if giants:
    if played_card == "witch":
        reward += weights.witch_behind_giant_bonus
        log.append(f"WITCH_BEHIND_GIANT(+{weights.witch_behind_giant_bonus:.1f})")
```

Figure 3. Reward assignment code from `reward.py` for the Giant push and Graveyard contextual bonuses.

L. Training Configuration

Training uses MaskablePPO from `sb3-contrib` [10] with the following hyperparameters.

Hyperparameter	Value	Rationale
<code>n_steps</code>	512	One gradient update per ~10-15 minutes of real gameplay; keeps learning within a single game rather than accumulating stale experience across many hours
<code>batch_size</code>	128	4 minibatches per update; larger than 64 to reduce gradient noise with the small <code>n_steps</code> buffer
<code>n_epochs</code>	10	Standard PPO reuse of rollout data

Hyperparameter	Value	Rationale
gamma	0.97	Effective horizon ~33 steps; better matches the 5-15 second delay between card play and tower damage than the default 0.99
gae_lambda	0.95	Standard GAE lambda
clip_range	0.2	Standard PPO clip
ent_coef	0.05 -> 0.01	Linear decay over training; high early entropy prevents collapse to WAIT spam, decays as strategy emerges
learning_rate	3e-4 -> 1e-5	Linear decay over 1,000,000 timesteps
vf_coef	0.5	Standard value function coefficient
max_grad_norm	0.5	Gradient clipping for stability
device	cpu	Intel Core i7, no discrete GPU

Table 4. MaskablePPO training hyperparameters.

Checkpoints are saved every 512 steps to the Models/checkpoints/ directory. TensorBoard logs are written to Models/tensorboard/. A background keyboard listener allows the user to press P to pause training between games (for menu navigation), S to save manually, and Q to quit and save.

Training was constrained to approximately 100 games due to the real-time gameplay requirement: each Clash Royale match lasts 3-4 minutes in BlueStacks, yielding roughly 6-8 hours of total training wall-clock time. At approximately 3-5 agent steps per second (limited by screen capture and YOLO inference on CPU), the model accumulated roughly 50,000 to 80,000 total timesteps.

V. Results

A. Computer Vision Subsystem Accuracy

Each subsystem was evaluated independently on held-out test data.

Subsystem	Metric	Result
LBGG Troop YOLO (YOLOv11s)	mAP50 on 10 test images	85%
SARG Spell YOLO (YOLOv11s)	mAP50 on 10 test images	92%
Elixir detection	Exact match on 20 test frames	100%
Hand detection	Correct 4-card identification on 20 test frames	100%
Tower health detection	Within 2% of ground truth on 22 test frames	91%

Table 5. Computer vision subsystem accuracy results.

The 85% troop detection accuracy reflects the challenge of small troop sprites at lower arena positions and visual similarity between units of the same type. The 9% tower health error rate is concentrated in frames immediately following a tower destruction animation, where the color of the bar transitions through several intermediate states before the tower registers as dead. The king bar confirmation mechanism (KING_CONFIRM_FRAMES = 3) was specifically designed to handle this case.

B. Agent Performance

The trained agent was evaluated in a series of 10 games against human opponents of varying skill levels, from approximately 4,000 trophies (intermediate) to 13,500 trophies (advanced competitive). The baseline comparison is a scripted deterministic bot (playing the same deck with hardcoded placement logic) evaluated against the same opponents.

Opponent Trophy Level	Games Played	ClashGPT Wins	Scripted Bot Wins
4,000	2	1	0
5,000	2	0	0
7,500	2	0	0
10,000	2	0	0
13,500	2	0	0
Total	10	1 (10%)	0 (0%)

Table 6. Agent win rate across opponent skill levels.

The single win was achieved in one of two games against a 4,000-trophy opponent, as shown in Table 6. No wins were recorded at any higher trophy level across the remaining eight games. The agent lost the left princess tower early (2-1 crown deficit), then successfully destroyed both of the opponent's princess towers to win 2-1 on crowns. This win demonstrates that the agent learned to mount a counterattack rather than simply spending elixir reactively. No wins were recorded against opponents at any other trophy levels.

C. Behavioral Observations

Several qualitatively interesting behaviors were observed during evaluation games, even in losses:

- The agent learned to play cards consistently rather than hoarding elixir, avoiding the elixir-capping penalty from the reward function.
- The agent placed Giants in the front of pushes on several occasions, consistent with the intended tank-and-support strategy of the deck archetype.
- The agent did not reliably learn to deploy Graveyard behind an active push, which would require recognizing the simultaneous conditions of (a) a Giant in the enemy half, (b) Graveyard being affordable, and (c) the correct timing. This is a higher-order strategy that likely requires substantially more training games to emerge.
- Against high-trophy opponents, the agent was exploited by aggressive early pushes: since it took several seconds to accumulate enough elixir for a 5-cost defensive unit, the opponent could destroy a princess tower before the agent had developed a response.

VI. Limitations

Several concrete limitations affect this work.

1. Training data volume: Only approximately 100 real-time games were used for training, yielding an estimated 50,000-80,000 timesteps. Comparable RL game agents such as OpenAI Five used on the order of tens of thousands of years of simulated gameplay [2]. The real-time constraint means that increasing training volume by 10x requires 10x more wall-clock time, making it infeasible to reach the sample counts typical of successful game AI without either a simulator or accelerated emulator.
2. Fixed deck and opponent: The agent was trained exclusively against one fixed opponent deck (Goblin Gang, Archer, Ice Spirit, Goblin Barrel, Rocket, Royal Hog, Log, Cannon with 3.0 average elixir). Because the YOLO models are also trained on the classes present in this matchup, the agent has no mechanism to observe or respond to cards outside its training distribution.
3. No temporal memory: The MLP policy in MaskablePPO has no recurrent component. Each decision is made solely from the current 202-dimensional observation. The agent cannot reason about opponent card cycling (inferring which of the 8 opponent cards are currently unavailable based on what was recently played) or anticipate incoming attacks based on historical patterns.
4. Tower health noise: The 9% tower health error rate, concentrated around the tower destruction animation window, can cause spurious reward signals. Although the multi-frame confirmation mechanism and the HP delta noise filter (0.5% threshold) reduce this, there are still frames during the game where the computed reward does not correspond to actual damage.
5. Troop kill rewards not implemented: The reward function includes infrastructure for rewarding enemy troop kills, but the implementation was not completed: the `prev_track_map` lookup needed to determine the team and class of a disappeared track was not wired through the environment. As a result, the agent has no direct incentive to spend elixir on area-damage cards to clear swarms.
6. Single-threaded inference: All computer vision, tracking, and reward computation runs synchronously on a single Intel Core i7 CPU thread. This limits the agent to approximately 3-5 steps per second, meaning it responds more slowly to rapidly evolving board states than a human player.

VII. Conclusion

This paper presented ClashGPT, the first fully open-source Clash Royale agent combining a complete computer vision pipeline with a trained reinforcement learning policy. The system correctly identifies troops, spells, elixir, hand contents, and all six tower health bars from raw pixels with high accuracy. After approximately 100 real-time training games, the agent achieves a 10% win rate against human opponents at the intermediate skill level, winning one game 2-1 after losing a tower, compared to a 0% win rate for a scripted baseline. This result satisfies the project hypothesis that a densely rewarded MaskablePPO agent can learn to play competitive Clash Royale within a tractable number of live training games.

The primary contribution of this work is a reproducible, modular pipeline that any researcher can extend. The code, model weights, training data, and setup instructions are available in the GitHub repository linked in this paper.

VIII. Future Work and Recommendations

Each item below directly addresses a specific limitation identified in Section VI.

7. Accelerated training via emulator speed-up: BlueStacks supports running at 2x game speed in some configurations. Enabling this would double training throughput without changing the architecture, potentially bringing the agent to 200 games of training in the same wall-clock time.
8. Recurrent policy: Replacing the MLP policy with a recurrent LSTM policy (available in sb3-contrib as RecurrentPPO) would allow the agent to track opponent card cycling over time. This is likely the single highest-impact architectural change for improving strategic play.
9. Complete troop kill reward: Wiring the `prev_track_map` through the environment to enable team-aware kill detection would give the agent a signal for using area spells and splash troops to clear swarms, which is a key defensive skill in Clash Royale.
10. Expanded YOLO training data: Adding labeled frames for all 121 cards and their evolution/hero variants would allow the agent to play and respond to any deck, rather than being restricted to the current eight-card training distribution.
11. Self-play curriculum: Training the agent against itself (with the opponent policy also controlled by the same model, or a lagged copy) would expose it to a broader range of strategies than a single fixed opponent deck and is the standard approach used in AlphaStar and OpenAI Five.
12. Opponent card tracking: Maintaining a probability distribution over the opponent's current hand based on observed plays would reduce the effective partial observability of the game and is a standard technique in card game AI.

References

- [1] [1] O. Vinyals et al., "Grandmaster level in StarCraft II using multi-agent reinforcement learning," *Nature*, vol. 575, pp. 350-354, Nov. 2019.
- [2] [2] C. Berner et al., "Dota 2 with large scale deep reinforcement learning," *arXiv preprint arXiv:1912.06680*, Dec. 2019.
- [3] [3] Ombarus, "Clash Royale AI Bot - Playing With Reinforcement Learning," YouTube, 2022. [Online]. Available: <https://www.youtube.com/watch?v=Q4cVkUFjvCo>. [Accessed: May 2026].
- [4] [4] Junior Builds, "I Made An AI That Plays Clash Royale," YouTube, 2023. [Online]. Available: <https://www.youtube.com/watch?v=6Gm-pnNieMU>. [Accessed: May 2026].
- [5] [5] ScreenShot (mss), "An ultra fast cross-platform multiple screenshots module in pure Python using ctypes," 2024. [Online]. Available: <https://python-mss.readthedocs.io/>.
- [6] [6] Label Studio, "Label Studio: Open source data labeling," 2024. [Online]. Available: <https://labelstud.io/>.
- [7] [7] G. Jocher, J. Qiu et al., "Ultralytics YOLO," Version 8.3.202, 2024. [Online]. Available: <https://ultralytics.com>.
- [8] [8] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple Online and Realtime Tracking," in *Proc. IEEE ICIP*, 2016.
- [9] [9] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.
- [10] [10] A. Raffin et al., "Stable-Baselines3: Reliable Reinforcement Learning Implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1-8, 2021.
- [11] [11] M. Towers et al., "Gymnasium," *Zenodo*, Mar. 2023. [Online]. Available: <https://zenodo.org/record/8127025>.
- [12] [12] TensorFlow, "Introduction to reinforcement learning with TensorFlow agents," 2024. [Online]. Available: https://www.tensorflow.org/agents/tutorials/0_intro_rl. [Accessed: Apr. 6, 2026].
- [13] [13] Wikipedia contributors, "Proximal policy optimization," *Wikipedia*, 2024. [Online]. Available: https://en.wikipedia.org/wiki/Proximal_policy_optimization. [Accessed: Mar. 16, 2026].
- [14] [14] OpenAI, "Proximal Policy Optimization," *Spinning Up in Deep RL*, 2018. [Online]. Available: <https://spinningup.openai.com/en/latest/algorithms/ppo.html>. [Accessed: Mar. 16, 2026].
- [15] [15] N. Zavodnik, "Clash Royale Troop Detection Dataset," *Roboflow Universe*, 2023. [Online]. Available: <https://universe.roboflow.com/nejc-zavodnik/clash-royale-troop-detection/dataset/13>. [Accessed: Feb. 13, 2026].
- [16] [16] wty-yy, "Clash-Royale-Detection-Dataset," *GitHub*, 2023. [Online]. Available: <https://github.com/wty-yy/Clash-Royale-Detection-Dataset>. [Accessed: Dec. 3, 2025].
- [17] [17] E. Gilchrest, "ClashGPT Demo Video Playlist," YouTube, May 2026. [Online]. Available: <https://www.youtube.com/playlist?list=PL32XXBfvd5bVLyjj0m9os1xYXtizypVCz>. [Accessed: May 2026].
- [18] [18] E. Gilchrest, "ClashGPT - The Autonomous Clash Royale Artificial Intelligence," *GitHub*, 2026. [Online]. Available:

<https://github.com/EzraGil08/ClashGPT-The-Autonomous-Clash-Royale-Artificial-Intelligence>. [Accessed: May 2026].

Appendix A: Software Versions and Hardware

Component	Version / Specification
Python	3.10.5
ultralytics (YOLOv11)	8.3.202
stable-baselines3	2.8.0
sb3-contrib (MaskablePPO)	2.8.0
gymnasium	per stable-baselines3 2.8.0 dependency
opencv-python	per ultralytics 8.3.202 dependency
filterpy (Kalman filter)	per tracker.py dependency
scipy (Hungarian matching)	per tracker.py dependency
pyautogui	used for click execution
mss	used for screen capture
CPU	Intel Core i7
RAM	15.6 GB
GPU	None (CPU training)
OS	Windows 10/11
Emulator	BlueStacks Android Emulator

Table A1. Software and hardware specifications for full reproducibility.

Appendix B: Action Space Layout

The 49 discrete actions are laid out as contiguous blocks per card in deck order, followed by the WAIT action at index 48. Table B1 shows the block boundaries.

Card	Elixir Cost	Num Placements	Action Indices
Giant	5	6	0 - 5
Bowler	5	6	6 - 11
Witch	5	6	12 - 17
Graveyard	5	2	18 - 19
Guards	3	6	20 - 25
Arrows	3	8	26 - 33
Snowball	2	8	34 - 41

Card	Elixir Cost	Num Placements	Action Indices
Minions	3	6	42 - 47
WAIT	-	1	48

Table B2. Complete action space layout with indices.

Appendix C: Repository Structure

The GitHub repository

(<https://github.com/EzraGil08/ClashGPT-The-Autonomous-Clash-Royale-Artificial-Intelligence>)

contains the following structure:

- `actions.py`: Action masking and execution via PyAutoGUI
- `constants.py`: Card metadata, placement coordinates, action space layout
- `env.py`: Gymnasium CEnv environment
- `hand_detection.py`: Template matching hand reader
- `misc_functions.py`: Anchor detection (`get_top_corner`) and elixir reading
- `observation.py`: 202-dimensional observation vector builder
- `reward.py`: Dense/sparse hybrid reward function
- `tracker.py`: SORT-based multi-object tracker with dual team pools
- `tower_health.py`: Pixel-scanning tower health detector
- `train.py`: MaskablePPO training script with banner detection and callbacks
- `debug_cv.py`: Live visualization overlay for debugging the CV pipeline
- `README.md`: Full setup, installation, and reproduction instructions